

NetBeans IDE Download Bundles

Supported technologies *	Java SE	Java EE	HTML5/JavaScript	PHP	C/C++	AI
NetBeans Platform SDK	•	•	•	•	•	•
Java SE	•	•	•	•	•	•
Java FX	•	•	•	•	•	•
Java EE	•	•	•	•	•	•
Java ME	•	•	•	•	•	•
HTML5/JavaScript	•	•	•	•	•	•
PHP	•	•	•	•	•	•
C/C++	•	•	•	•	•	•
Grails	•	•	•	•	•	•
Java C#™ 3 Connected Bundles	•	•	•	•	•	•
GlassFish Server Open Source Edition 4.1.1	•	•	•	•	•	•
Apache Tomcat 8.0.27	•	•	•	•	•	•
	Download	Download	Download .deb	Download .deb	Download .deb	Download
	Free, 35 MB	Free, 102 MB	Free, 104 - 107 MB	Free, 104 - 107 MB	Free, 104 - 110 MB	Free, 2.15 MB

Figure 1: NetBeans' download page

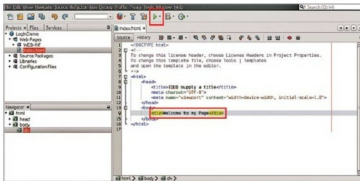


Figure 2: Project view

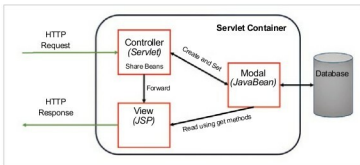


Figure 3: The model-view-controller architecture

Installing on Windows and Mac systems is straightforward. Just click on the downloaded file, i.e., `exe` for Windows and `dmg` for Mac, and follow the onscreen instructions.

For installing in Linux, navigate to the downloaded directory, find the `sh` file, and change the permission of the `sh` file to executable using the `chmod` command, as follows:

```
chmod +x filename.sh
```

Replace the file name with the downloaded file name. Execute the shell script using the following command:

```
./filename.sh
```

While downloading, GlassFish Server and Apache Tomcat Server will get installed for running Web applications.

Creating a Web application in NetBeans

When NetBeans is started for the first time, the default startup page will get displayed. For this article, we will use the GlassFish server and JavaDB RDBMS as these come preinstalled and preconfigured. We can easily configure any other server like Tomcat, JBoss, WebSphere, WebLogic, etc. In the same way, any database can be used for development.

To start developing a Web application, click on the **File** menu, select **New Project**, then **Java Web** under **Categories** and, finally, select **Web Application** from **Projects**.

NetBeans will activate Java Web and EE, which may take a few seconds. Once done, a **Name** and **Location** option will appear, which requires the project's name, location and the project folder.

Specify the project's name, leave the rest as default and click **Next**. Server information will be displayed, which by default is GlassFish server. We can see the Java EE version and the context path of the application, which is the same as the project name.

Clicking on **Next** will display the options of different frameworks that can be used in Web application; next, click **Finish**. We can now see the view in the **Project Window** on the left pane; on the right side, we can see the source code editor, which will display the content of `index.html` file. I changed the body content of `index.html` to `Welcome to my page`. To run the application, click on the green arrow button on the toolbar as shown in Figure 2.

We can also run the application by right-clicking on the project name in the **Project Window** and selecting the **Run** option. Below the code editor, the output panel will be displayed, showing the status of the GlassFish server, application and JavaDB. Once completed, the browser will open and the output of `index.html` will be displayed.

In the address bar of the browser, you will find localhost, which is from the Web server running on the local machine; in our case it is GlassFish and our application or project name is displayed. `Index.html` is the default file that the Web server will send whenever the request for the application comes.

What is MVC?

MVC stands for model-view-controller. MVC is a design pattern that separates the data, presentation and action part. In very simple terms, the controller acts as an interface between model and view, intercepting all incoming requests. Model represents an object that carries the state of an application, i.e., data. View represents the user interface, i.e., the presentation of the data. In Java Web applications, Servlet acts as the controller, JSP acts as view and Java Beans or POJO acts as a model, as shown in Figure 3.